```
"""
Gemini API 财报分析器
支持 PDF 直传，输出标准化 JSON 结构
"""

import json
import time
import logging
from pathlib import Path
from typing import Optional

import google.generativeai as genai

logger = logging.getLogger(__name__)


# ── 分析模板 ──────────────────────────────────────────────────────────
ANALYSIS_PROMPT = """
你是一位专业的A股财务分析师，专注于中国上市公司深度研究。

请仔细阅读这份年度报告，严格按以下JSON格式输出分析结果。
不要输出任何JSON以外的内容，不要加markdown代码块标记。

{
  "基本信息": {
    "公司名称": "",
    "股票代码": "",
    "报告期": "",
    "审计意见": ""
  },
  "核心财务数据": {
    "营收":        {"金额亿元": 0.0, "同比增长率": ""},
    "净利润":      {"金额亿元": 0.0, "同比增长率": ""},
    "扣非净利润": {"金额亿元": 0.0, "同比增长率": ""},
    "毛利率":      "",
    "净利率":      "",
    "ROE":         "",
    "经营现金流": {"金额亿元": 0.0},
    "合同负债":    {"金额亿元": 0.0, "同比增长率": ""},
    "有息负债":    {"金额亿元": 0.0},
    "资产负债率": ""
  },
  "现金流质量": {
    "经营现金流_净利润比": "",
```

```
    "判断": ""
  },
  "关键变化": [
    "变化点1",
    "变化点2",
    "变化点3"
  ],
  "管理层核心表述": [
    "表述1",
    "表述2"
  ],
  "订单与在手合同": {
    "在手订单": "",
    "新签合同": "",
    "合同负债分析": ""
  },
  "风险因素": [
    "风险1",
    "风险2"
  ],
  "估值参考": {
    "EPS": "",
    "当前PE区间参考": "需结合市价计算",
    "PB": ""
  },
  "分析师结论": "",
  "值得深挖的问题": [
    "问题1",
    "问题2"
  ]
}
"""


class ReportAnalyzer:
    """
    Gemini API财报分析器

    使用方法:
        analyzer = ReportAnalyzer(api_key='xxx', model='gemini-1.5-flash')
        result = analyzer.analyze('reports/600893.SH/2024_annual_report.pdf')
        results = analyzer.batch_analyze({'600893.SH_2024': Path(...)})
    """

    def __init__(
        self,
        api_key: str,
```

```
    model: str = "gemini-1.5-flash",   # flash更便宜，pro效果更好
    sleep_between: float = 5.0,
    max_retries: int = 3,
):
    genai.configure(api_key=api_key)
    self.model = genai.GenerativeModel(model)
    self.sleep_between = sleep_between
    self.max_retries = max_retries

# ------------------------------------------------------------------
# 公开接口
# ------------------------------------------------------------------

def analyze(self, pdf_path: str | Path) -> Optional[dict]:
    """分析单份财报PDF，返回结构化dict"""
    pdf_path = Path(pdf_path)
    if not pdf_path.exists():
        logger.error(f"PDF不存在: {pdf_path}")
        return None

    for attempt in range(1, self.max_retries + 1):
        try:
            logger.info(f"[Gemini] 分析 {pdf_path.name} (尝试 {attempt}/{self.r
            result = self._call_gemini(pdf_path)
            logger.info(f"[ok] {pdf_path.name} 分析完成")
            return result
        except json.JSONDecodeError as e:
            logger.warning(f"JSON解析失败(尝试{attempt}): {e}")
        except Exception as e:
            logger.warning(f"Gemini调用失败(尝试{attempt}): {e}")

        if attempt < self.max_retries:
            time.sleep(self.sleep_between * attempt)

    logger.error(f"[fail] {pdf_path.name} 分析失败，已重试{self.max_retries}次")
    return None

def batch_analyze(
    self,
    pdf_map: dict[str, Path],   # {'600893.SH_2024': Path(...)}
) -> dict[str, dict]:
    """批量分析，返回 {key: result_dict}"""
    results = {}
    total = len(pdf_map)
    for i, (key, path) in enumerate(pdf_map.items(), 1):
        logger.info(f"[{i}/{total}] 处理 {key}")
        result = self.analyze(path)
```

```
        if result:
            results[key] = result
        time.sleep(self.sleep_between)
    return results

# ---------------------------------------------------------------------
# 内部方法
# ---------------------------------------------------------------------

def _call_gemini(self, pdf_path: Path) -> dict:
    with open(pdf_path, "rb") as f:
        pdf_bytes = f.read()

    response = self.model.generate_content([
        {
            "mime_type": "application/pdf",
            "data": pdf_bytes,
        },
        ANALYSIS_PROMPT,
    ])

    raw = response.text.strip()
    # 清理可能残留的markdown代码块标记
    raw = raw.removeprefix("```json").removeprefix("```").removesuffix("```")
    return json.loads(raw)
```